Understanding the Sensor Node Hardware

Aim: To understand the basics of various sensor nodes

Theory:

Introduction:

Sensor nodes are fundamental components in modern technology, facilitating the collection of data in various applications, ranging from environmental monitoring to industrial automation. Understanding the basics of sensor nodes is crucial for undergraduate students, as it lays the foundation for their comprehension of advanced concepts in sensor networks. This article aims to provide a comprehensive overview of sensor nodes, their types, functionalities, and applications.

What is a Sensor Node?

A sensor node, also known as a sensor mote or a smart sensor, is a small electronic device equipped with sensors, a processing unit, and communication capabilities. These nodes are designed to gather data from their surrounding environment and transmit it to a central processing unit or a data collection point. The key components of a sensor node include sensors, microcontrollers, transceivers, power sources, and memory.

Terms related to Sensor node

1. Sensor:
In the context of Wireless Sensor Networks (WSNs), a sensor is a device or component that detects physical or environmental changes such as temperature, humidity, light intensity, or motion. These changes are converted into electrical signals which are then processed for further analysis or transmitted wirelessly to a central location for monitoring and analysis. Sensors in WSNs are typically small, low-power devices designed to operate autonomously or as part of a network, contributing to data collection and monitoring tasks in various applications such as environmental monitoring, healthcare, and industrial automation.

2. Node:
In a Wireless Sensor Network (WSN), a node refers to any device or element that is part of the network. This includes sensor nodes, which are equipped with sensors for data collection, as well as other types of nodes such as relay nodes, router nodes, and gateway nodes. Sensor nodes are the primary components responsible for sensing the environment, collecting data, processing information, and communicating with other nodes within the network. Each node in a WSN typically has its unique identifier and may perform specific functions such as data aggregation, routing, or data forwarding to ensure efficient communication and data transmission within the network.

3. Base station GUI:
The Base Station GUI (Graphical User Interface) is a software interface designed to facilitate communication between the user or operator and the base station in a Wireless Sensor Network (WSN). The base station serves as a central point for collecting data from sensor nodes deployed in the field. The GUI provides a user-friendly platform for monitoring, configuring, and managing the WSN, allowing users to visualize data, set thresholds, configure parameters, and perform various administrative tasks. Through the base station GUI, users can access real-time or historical data collected by sensor nodes, analyze sensor readings, and make informed decisions based on the information provided by the WSN. The GUI interface may include features such as data visualization tools, alarm notifications, network topology displays, and configuration wizards to simplify the operation and management of the WSN.

Types of Sensor Nodes:

1. Passive Sensor Nodes: These nodes do not require a power source and operate by detecting changes in the surrounding environment. Examples include passive infrared (PIR) sensors used in motion detection systems.

2. Active Sensor Nodes: Active sensor nodes are powered devices equipped with sensors and processing units. They actively collect and process data before transmitting it wirelessly. Examples include temperature sensors, humidity sensors, and gas sensors.

3. Wireless Sensor Nodes: These nodes communicate wirelessly with other nodes or a central hub, making them suitable for distributed sensing applications. They are commonly used in environmental monitoring, smart agriculture, and smart cities.

4. Wired Sensor Nodes: Wired sensor nodes are connected to a central system via cables. They are often used in industrial settings where reliability and stability are critical.

5. Hybrid Sensor Nodes: Hybrid sensor nodes combine features of both wireless and wired nodes, offering flexibility and robustness in data collection and transmission.

Functionality of Sensor Nodes:

- Sensing: Sensor nodes detect physical, chemical, or environmental changes using various types of sensors such as temperature sensors, pressure sensors, and accelerometer sensors.

- Processing: The collected data is processed locally within the sensor node using microcontrollers or digital signal processors (DSPs). This processing may include filtering, calibration, or data compression.

- Communication: Sensor nodes transmit the processed data to a central processing unit or a data collection point using wired or wireless communication protocols such as Wi-Fi, Bluetooth, Zigbee, or LoRaWAN.

- Power Management: Efficient power management is essential to prolong the lifespan of sensor nodes. Power can be supplied through batteries, solar panels, or energy harvesting techniques.

Applications of Sensor Nodes:

1. Environmental Monitoring: Sensor nodes are used to monitor air quality, water quality, and soil conditions in environmental monitoring projects.

2. Smart Agriculture: In agriculture, sensor nodes are deployed to monitor soil moisture, temperature, and humidity levels, enabling farmers to optimize irrigation and crop management practices.

3. Industrial Automation: Sensor nodes play a crucial role in industrial automation by monitoring equipment performance, detecting faults, and ensuring workplace safety.

4. Healthcare: In healthcare applications, sensor nodes are utilized for remote patient monitoring, fall detection systems, and monitoring vital signs such as heart rate and blood pressure.

5. Smart Cities: Sensor nodes are deployed in smart city projects to monitor traffic flow, manage energy consumption, and enhance public safety through surveillance systems.

Some typical Sensors types

1) Mechanical Sensors

A. Strain Gauge Sensor:
Strain gauge sensors measure how much something stretches or squashes. They work by using a thin wire that changes its resistance when stretched or compressed. These sensors are used in things like checking if bridges are safe or in weighing scales.

B. Accelerometer:
Accelerometers measure how fast something is speeding up or slowing down. They work by using a small weight that moves when there's acceleration. These sensors are in phones to flip the screen when you turn it and in cars to help airbags deploy when there's a crash.

These sensors are important because they help us understand and control movement and forces in various objects and systems.

2) Humidity Sensor:
Humidity sensors are electronic devices that measure and monitor the moisture content in the air or gas. They play a crucial role in maintaining optimal humidity levels in various environments, ensuring comfort, safety, and efficiency.
These sensors work based on different principles, such as capacitive, resistive, or thermal conductivity. The most common type is the capacitive humidity sensor, which detects changes in capacitance as humidity levels fluctuate.
Humidity sensors find applications in HVAC systems, industrial processes, weather monitoring, agriculture, medical devices, and consumer electronics. They enable precise control of humidity levels, contributing to improved indoor air quality, product quality, and environmental conditions.
In essence, humidity sensors are essential tools for measuring and managing moisture levels, enhancing comfort, health, and productivity in diverse settings.

3) Temperature Sensor:

Temperature sensors are indispensable devices used to measure the temperature of their surroundings. They play a crucial role in various applications where temperature monitoring is essential for controlling processes, ensuring safety, and optimizing performance.

These sensors operate based on different principles, such as resistance, voltage, or thermoelectric effects, to convert temperature changes into electrical signals. Common types of temperature sensors include thermocouples, thermistors, and resistance temperature detectors (RTDs).

Temperature sensors find applications in HVAC systems, automotive engines, weather monitoring, medical equipment, and process control. Their accurate and reliable measurements enable precise control and monitoring, contributing to improved efficiency, safety, and performance in various industries.

In essence, temperature sensors are vital tools for measuring and monitoring temperature variations in different environments and applications, facilitating optimal operation and control.

4) Pressure Sensor:

Pressure sensors are essential components used to measure the force exerted by a fluid on their sensing elements. They operate based on principles like piezoresistivity, capacitive sensing, or piezoelectricity.

These sensors find applications in diverse fields, including automotive systems, industrial automation, aerospace, and medical devices. They are

crucial for monitoring and controlling processes, ensuring safety, and optimizing performance.

Pressure sensors provide accurate and reliable measurements, enabling precise control and monitoring in various industries. Their versatility and efficiency contribute to improved efficiency, safety, and performance in different applications.

In summary, pressure sensors are vital tools for measuring and monitoring pressure variations in different environments and applications, facilitating optimal operation and control.

Exploring and understanding TinyOS computational concepts

Aim: To understand Events, Commands and Task, nesC model, nesC Components

Theory:

1) Events:

   In the context of operating systems (OS) used in Wireless Sensor Networks (WSN), events are occurrences or notifications triggered by hardware or software components. These events represent significant changes in the system's state or external environment and serve as triggers for executing specific actions or tasks. In TinyOS, an event-driven operating system commonly used in WSN, events play a crucial role in coordinating the execution of tasks and handling asynchronous events such as sensor readings or network events. Programmers define event handlers, which are functions responsible for responding to specific events by executing predefined actions or invoking other components. Events enable efficient and asynchronous communication between various components of the system, facilitating modular and scalable application development in WSNs.

2) Commands and Tasks:

   In the context of TinyOS, commands and tasks are fundamental constructs used for defining and executing operations within the operating system. Commands represent synchronous operations that are executed immediately when invoked, typically performing actions such as configuring hardware components or initiating data transmission. Tasks, on the other hand, represent asynchronous operations that are scheduled for execution at a later time or in response to specific events. Tasks are often used for performing time-consuming operations or handling asynchronous events such as sensor readings or network events. TinyOS provides mechanisms for defining commands and tasks within components, allowing developers to implement efficient and responsive applications for WSNs.

3) nesC Model:

   The nesC (network embedded systems C) model is a programming model and language specifically designed for developing embedded systems and WSN applications. It is closely associated with TinyOS, the operating system commonly used in WSNs. The nesC model emphasizes modularity,

concurrency, and resource efficiency, enabling developers to build scalable and robust applications for resource-constrained environments. Key features of the nesC model include a component-based architecture, event-driven programming, and support for concurrency through lightweight tasks and message passing. NesC provides abstractions for programming sensor nodes, allowing developers to focus on application logic rather than low-level system details. It promotes code reuse, modularity, and portability, facilitating the development of complex WSN applications with minimal resource overhead.

4) nesC Components:

In the nesC programming model, components are modular building blocks used to encapsulate functionality and promote code reuse in WSN applications. Components represent reusable units of code that encapsulate specific functionality, such as sensor drivers, communication protocols, or application logic. They define interfaces for interacting with other components and can be composed hierarchically to create complex systems. Components communicate with each other through well-defined interfaces using events, commands, and tasks. NesC components promote modularity, scalability, and maintainability in WSN applications by allowing developers to break down complex systems into smaller, manageable units. They facilitate code reuse, enable rapid prototyping, and support modular design practices, making it easier to develop and maintain WSN applications.

5) TOSSIM stands for TinyOS Simulator. It is a discrete-event simulator designed specifically for TinyOS, the operating system commonly used in Wireless Sensor Networks (WSNs). TOSSIM allows developers to simulate the behaviour of TinyOS-based applications and the underlying network on a computer without the need for physical sensor nodes.

Key features of TOSSIM include:

a. Accuracy: TOSSIM provides an accurate simulation environment that mimics the behaviour of TinyOS applications and the WSN network, allowing developers to test their code and algorithms in a controlled environment.
b. Efficiency: TOSSIM is optimized for efficiency, allowing developers to simulate large-scale WSNs with hundreds or even thousands of sensor nodes on a standard computer.
c. Debugging: TOSSIM includes debugging tools and features that enable developers to monitor and analyse the behaviour of their applications during simulation, helping to identify and debug errors and optimize performance.

d. Flexibility: TOSSIM is highly flexible and customizable, allowing developers to configure various parameters such as network topology, node behaviour, and communication protocols to simulate specific scenarios and test different use cases.

Exploring and understanding TinyOS computational concepts

Aim: To Understand TOSSIM for Mote-mote radio communication and Mote-PC serial communication

Theory:

TOSSIM:

TOSSIM stands for TinyOS Simulator. It is a discrete-event simulator designed specifically for TinyOS, the operating system commonly used in Wireless Sensor Networks (WSNs). TOSSIM allows developers to simulate the behaviour of TinyOS-based applications and the underlying network on a computer without the need for physical sensor nodes.

Key features of TOSSIM include:

a. Accuracy: TOSSIM provides an accurate simulation environment that mimics the behaviour of TinyOS applications and the WSN network, allowing developers to test their code and algorithms in a controlled environment.
b. Efficiency: TOSSIM is optimized for efficiency, allowing developers to simulate large-scale WSNs with hundreds or even thousands of sensor nodes on a standard computer.
c. Debugging: TOSSIM includes debugging tools and features that enable developers to monitor and analyse the behaviour of their applications during simulation, helping to identify and debug errors and optimize performance.

d. Flexibility: TOSSIM is highly flexible and customizable, allowing developers to configure various parameters such as network topology, node behaviour, and communication protocols to simulate specific scenarios and test different use cases.

Mote:

A "mote" refers to a small, low-power, autonomous sensor node equipped with sensing, processing, and communication capabilities. Motes are the fundamental building blocks of WSNs and are typically deployed in large numbers to monitor and collect data from the physical environment.

Key characteristics of motes include:

1) Small Size: Motes are compact and lightweight, making them easy to deploy in various environments, including remote or hard-to-reach locations.
2) Low Power: Motes are designed to operate on battery power or energy harvesting techniques, such as solar panels or vibration energy harvesters, to prolong their lifespan and reduce maintenance requirements.
3) Sensing Capabilities: Motes are equipped with various types of sensors, such as temperature, humidity, light, motion, and gas sensors, to collect data about the surrounding environment.
4) Processing Power: Despite their small size, motes contain microcontrollers or low-power processors capable of processing sensor data and executing application-specific algorithms.
5) Communication: Motes communicate wirelessly with each other or with a central base station using radio frequency (RF) communication protocols. This enables data exchange and coordination among sensor nodes within the network.
6) Autonomy: Motes are often autonomous and operate without human intervention once deployed. They can perform tasks such as data collection, processing, and communication independently or in coordination with other motes in the network.

TOSSIM for Mote-Mote Radio Communication:

TOSSIM for Mote-Mote Radio Communication refers to the capability of the TOSSIM simulator to emulate wireless communication between sensor nodes, often referred to as motes, in a Wireless Sensor Network (WSN) environment.

In WSNs, sensor nodes communicate with each other wirelessly using radio frequency (RF) signals. TOSSIM simulates this communication by modelling the behaviour of RF transmissions and receptions between virtual sensor nodes within the simulated environment.

Key aspects of TOSSIM for Mote-Mote Radio Communication include:

1) Radio Model: TOSSIM includes a radio model that simulates the transmission and reception of RF signals between sensor nodes. This model accounts for factors such as signal strength, interference, and packet loss, providing an accurate representation of real-world wireless communication.
2) Packet Transmission: Sensor nodes in TOSSIM can send and receive packets over the simulated radio channel. Developers can specify

parameters such as packet size, transmission power, and data rate to configure the behaviour of packet transmission in the simulation.

3) Interference and Channel Effects: TOSSIM simulates the effects of interference and channel conditions on wireless communication. This includes modelling factors such as multi-path propagation, fading, and noise, which can impact the reliability and performance of communication between sensor nodes.
4) MAC Protocols: TOSSIM supports the simulation of various Medium Access Control (MAC) protocols used in WSNs, such as TDMA (Time Division Multiple Access) and CSMA (Carrier Sense Multiple Access). These protocols govern how sensor nodes access the shared radio channel and coordinate their communication to avoid collisions and contention.

Mote-PC serial communication

Mote-PC serial communication refers to the exchange of data between a mote, which is a sensor node typically used in Wireless Sensor Networks (WSNs), and a personal computer (PC) using serial communication protocols. This communication allows the PC to interact with and gather data from the mote, enabling monitoring, control, and analysis of the sensor data collected by the mote.

Key components and aspects of mote-PC serial communication include:

1. Serial Ports: Both the mote and the PC must have serial ports (or virtual serial ports) to establish a communication link. Serial ports provide a physical interface for transmitting and receiving serial data using UART (Universal Asynchronous Receiver-Transmitter) or USB-to-serial converters.

2. Serial Communication Protocols: Serial communication between the mote and the PC typically uses standard protocols such as RS-232, RS-485, or USB CDC (Communication Device Class). These protocols define the format and timing of data transmission, including baud rate, data bits, parity, and stop bits.

3. Communication Libraries or APIs: Developers often use communication libraries or application programming interfaces (APIs) to facilitate mote-PC serial communication. These libraries provide functions and methods for opening, configuring, sending, and receiving data over the serial port from both the mote and the PC.

4. Data Exchange: Data exchanged between the mote and the PC can include sensor readings, configuration parameters, commands, and status updates. The data format may vary depending on the application and the specific requirements of the mote and PC software.

5. Software Interfaces: On the PC side, software applications or scripts are used to communicate with the mote via the serial port. These applications may include custom-developed software, terminal emulators, or serial communication tools that enable users to interact with the mote and process the data received.

6. Data Processing and Analysis: Once data is received from the mote, the PC can process and analyze it using various software tools and algorithms. This may involve data visualization, statistical analysis, machine learning, or integration with other systems for further processing or decision-making.

Create and simulate a simple ad hoc network

Aim:
To create a simple ad hoc network and perform its simulation with the required number of hosts

Software Used:
Omnetpp 5.7, INET 4.3.6 framework

Theory:
An ad hoc network is one that is spontaneously formed when devices connect and communicate with each other.
Ad hoc networks are mostly wireless local area networks (LANs).
The devices communicate with each other directly instead of relying on a base station or access points as in wireless LANs for data transfer co-ordination.
Each device participates in routing activity, by determining the route using the routing algorithm and forwarding data to other devices via this route.

Types of Wireless Ad Hoc Networks
Wireless ad hoc networks are categorized into classes. Here are a few examples:

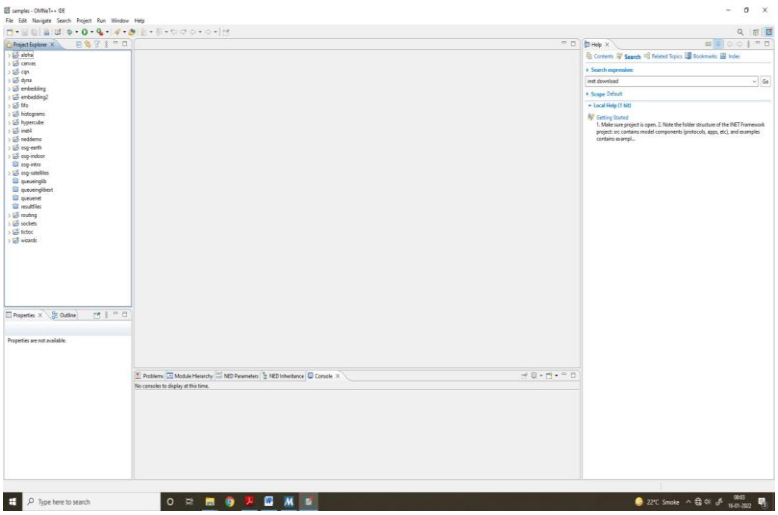
1) Mobile ad hoc network (MANET): An ad hoc network of mobile devices.
2) Vehicular ad hoc network (VANET): Used for communication between vehicles. Intelligent VANETs use artificial intelligence and ad hoc technologies to communicate what should happen during accidents.
3) Smartphone ad hoc network (SPAN): Wireless ad hoc network created on smartphones via existing technologies like Wi-Fi and Bluetooth.
4) Wireless mesh network: A mesh network is an ad hoc network where the nodes communicate directly with each other to relay information throughout the network.
5) Army tactical MENT: Used in the army for "on-the-move" communication, a wireless tactical ad hoc network relies on range and instant operation to establish networks when needed.
6) Wireless sensor network: Wireless sensors that collect everything from temperature and pressure readings to noise and humidity levels can form an ad hoc network to deliver information to a home base without needing to connect directly to it.
7) Disaster rescue ad hoc network: Ad hoc networks are important when disaster strikes and established communication hardware isn't functioning properly.

Limitations of Ad Hoc Wireless Network

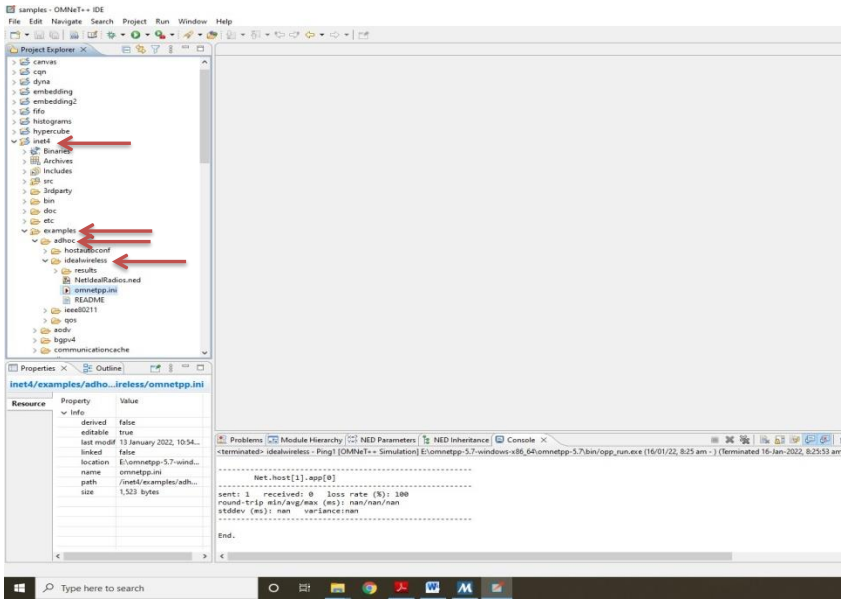
1) For file and printer sharing, all users need to be in the same workgroup, or if one computer is joined to a domain, the other users must have accounts on that computer to access shared items.
2) Other limitations of ad hoc wireless networking include the lack of security and a slow data rate. Ad hoc mode offers minimal security; if attackers come within range of your ad hoc network, they won't have any trouble connecting.

We create an Ad hoc network with 7 hosts using Omnetpp and INET through the following steps

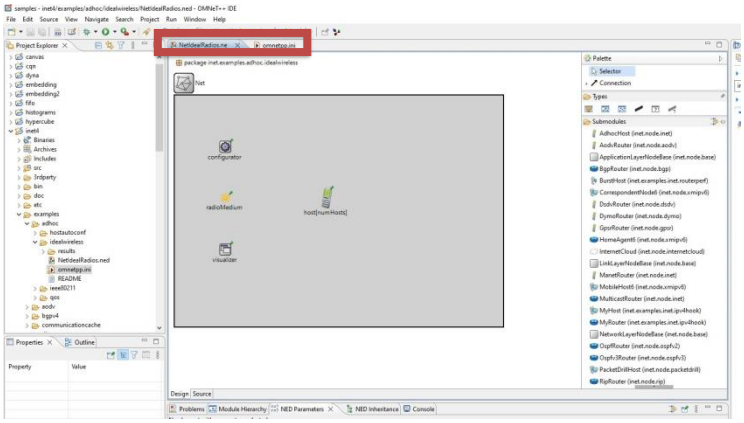
Step 1: The omnetpp simulator user interface



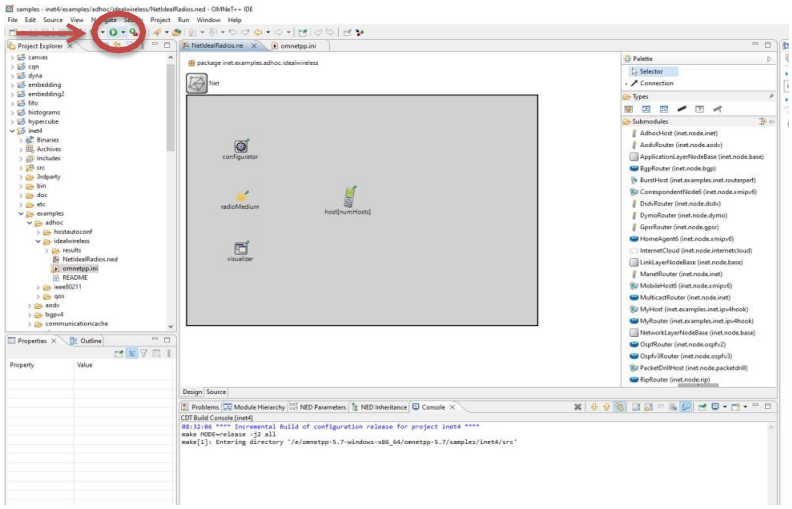
Step 2: Click on inet folder, then in it click on examples, then on adhoc and then on idealwireless as given



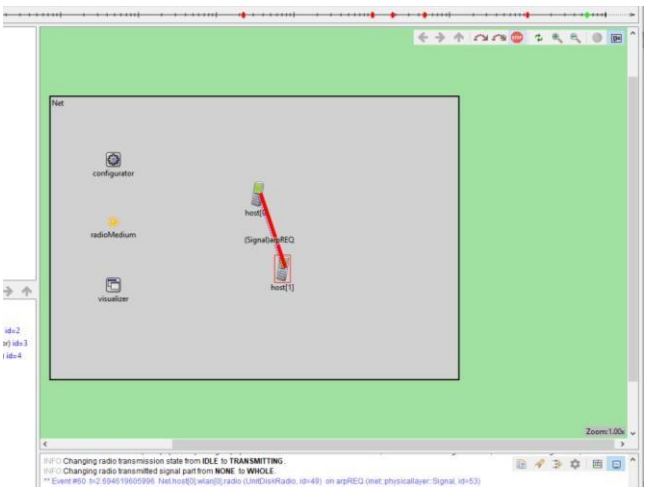
Step 3: In order to load the simulation, double click on two files NetIdealRadios.ned and omnetpp.ini



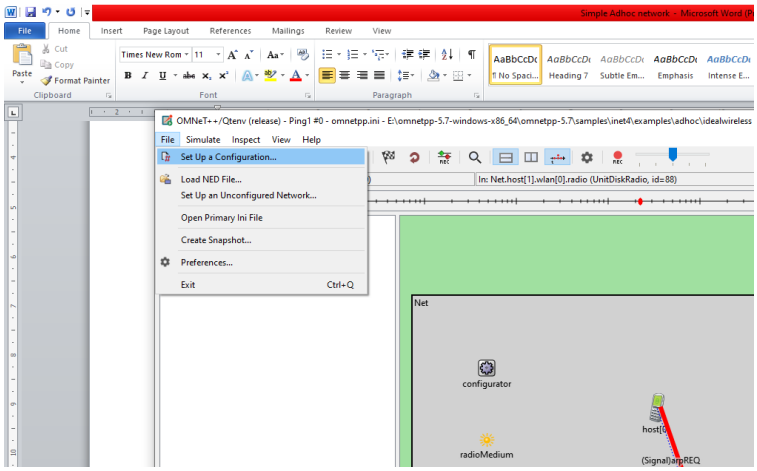
Step 4: Now we run the simulation



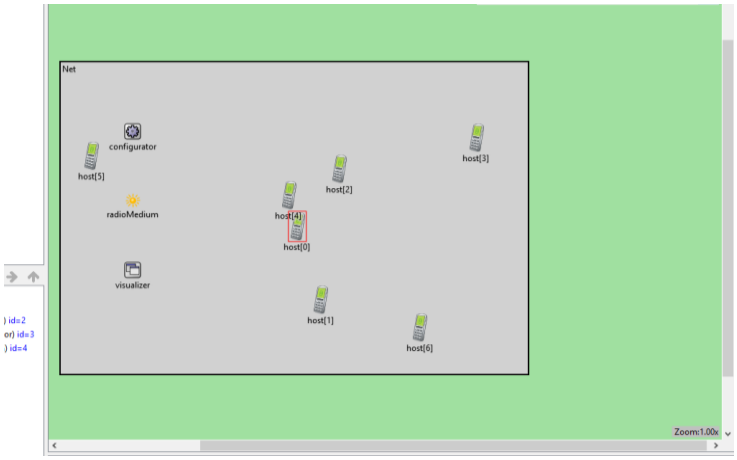
Step 5: After running the simulation we get the following



The number of hosts can be increased by the following



In this we get a dropdown menu, select n host option and enter the required hosts
The following simulation has 7 hosts



For video demonstration of the given practical click on the link below or Scan the QR-code

https://youtu.be/Y0gC-jrDm1E

Understanding, Reading and Analyzing Routing Table of a network

Aim:

To create a network and set the routing path, understand and analyze the routing table of the networks

Software Used:

Cisco Packet Tracer 7.2.0.026

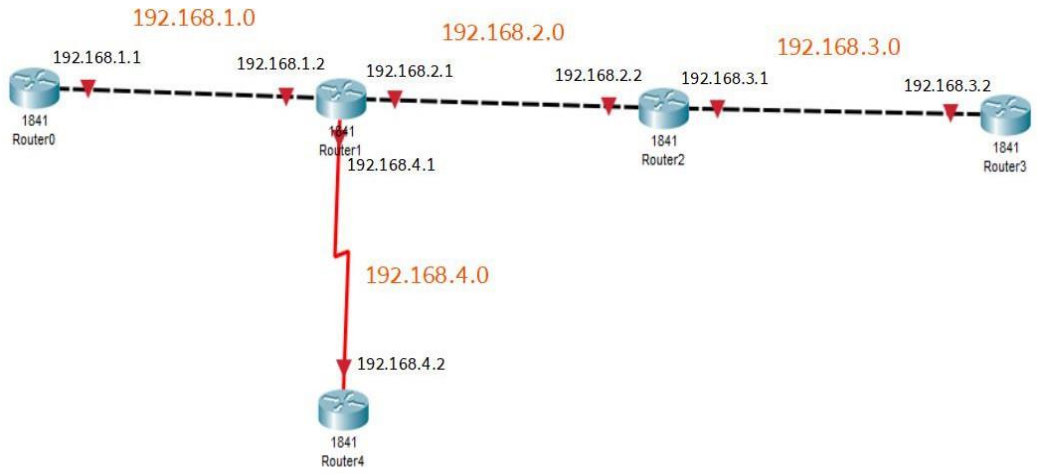
Theory:

A routing table is similar to a distribution map in package delivery. Whenever a node needs to send data to another node on a network, it must first know where to send it.
If the node cannot directly connect to the destination node, it has to send it via other nodes along a route to the destination node.
Each node needs to keep track of which way to deliver various packages of data, and for this it uses a routing table.
A routing table is a database that keeps track of paths, like a map, and uses these to determine which way to forward traffic.
A routing table is a data file in RAM that is used to store route information about directly connected and remote networks. Nodes can also share the contents of their routing table with other nodes.

The primary function of a router is to forward a packet towards its destination network, which is the destination IP address of the packet.
To do this, a router needs to search the routing information stored in its routing table.
The routing table contains network/next hop associations. These associations tell a router that a particular destination can be optimally reached by sending the packet to a specific router that represents the next hop on the way to the final destination. The next hop association can also be the outgoing or exit interface to the final destination.
An example of Routing Table

Network destination	Netmask	Gateway	Interface	Metric
0.0.0.0	0.0.0.0	192.168.0.1	192.168.0.100	10
127.0.0.0	255.0.0.0	127.0.0.1	127.0.0.1	1
192.168.0.0	255.255.255.0	192.168.0.100	192.168.0.100	10
192.168.0.100	255.255.255.255	127.0.0.1	127.0.0.1	10
192.168.0.1	255.255.255.255	192.168.0.100	192.168.0.100	10

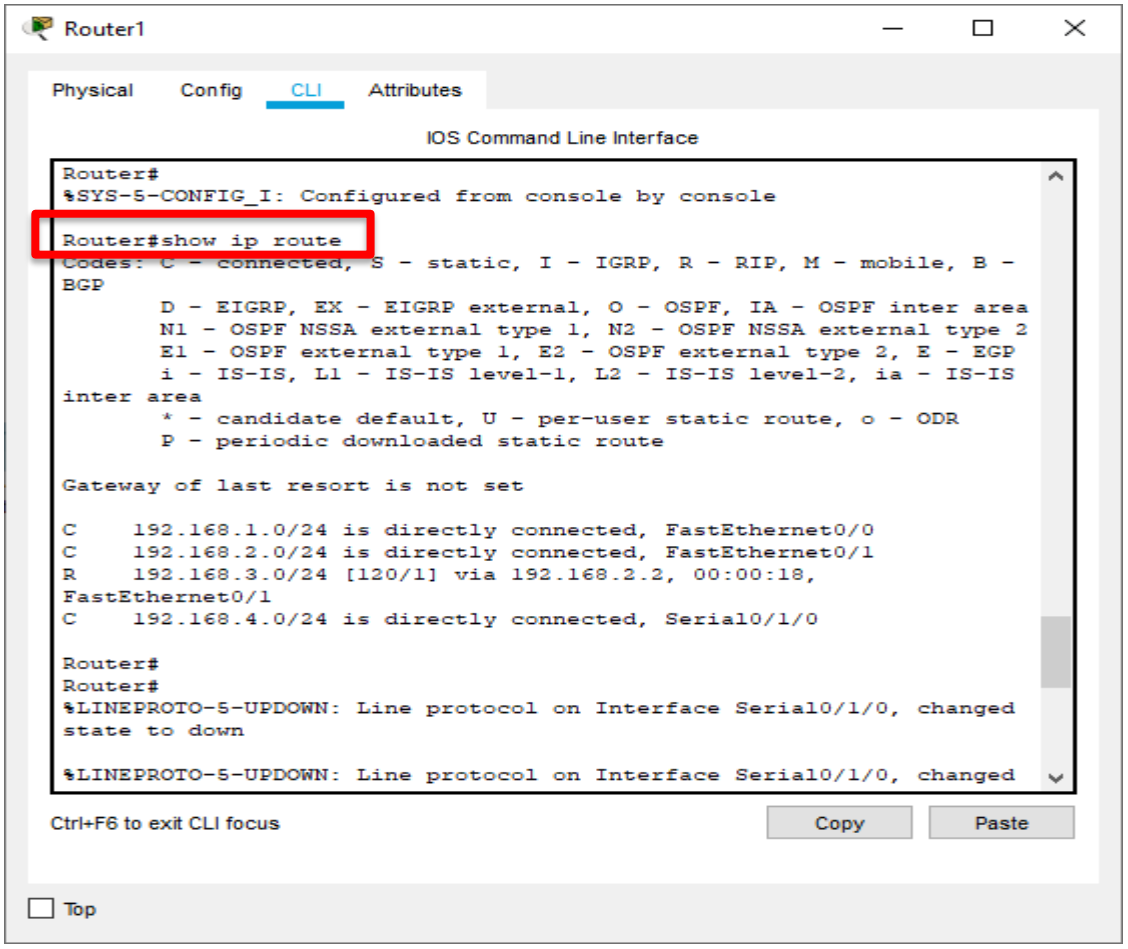
Consider the following topology



The ip addresses are configured on the given interfaces of the Routers.

The Routing path is also set using RIP.

The following command is executed in the CLI mode of Router1



We get the following Routing information from Router1

C 192.168.1.0/24 is directly connected, FastEthernet0/0
C 192.168.2.0/24 is directly connected, FastEthernet0/1
R 192.168.3.0/24 [120/1] via 192.168.2.2, 00:00:18, FastEthernet0/1
C 192.168.4.0/24 is directly connected, Serial0/1/0

The above is the required output

For the video demonstration of the above practical click on the link below or scan the QR-code

https://youtu.be/mGu_FzEVTiA

MANET implementation simulation

Aim:

To create a basic MANET implementation simulation for Packet animation and Packet Trace

Software Used:

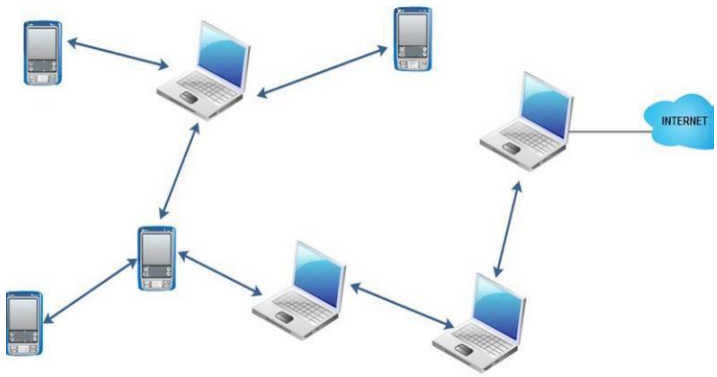
Omnet++ 5.7, INET 4.3.6 framework

Theory:

1) MANET (Mobile Adhoc NETwork) also called a wireless Adhoc network or Adhoc wireless network that usually has a routable networking environment on top of a Link Layer ad hoc network
2) They consist of a set of mobile nodes connected wirelessly in a self-configured, self-healing network without having a fixed infrastructure.
3) MANET nodes are free to move randomly as the network topology changes frequently.
4) Each node behaves as a router as they forward traffic to other specified nodes in the network.
5) MANET may operate a standalone fashion or they can be part of larger internet.
6) They form a highly dynamic autonomous topology with the presence of one or multiple different transceivers between nodes.
7) The main challenge for the MANET is to equip each device to continuously maintain the information required to properly route traffic.

The following are some of the important characteristics of MANET

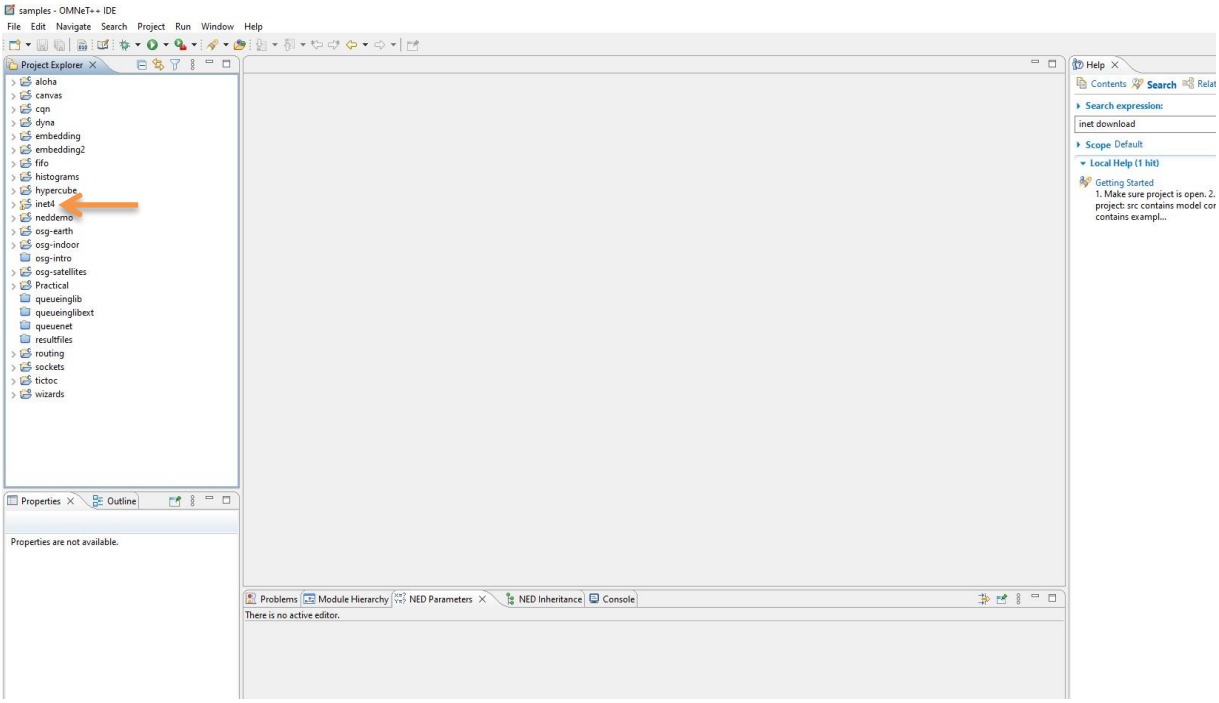
1) Dynamic Topologies
2) Bandwidth constrained, variable capacity links:
3) Autonomous Behavior:
4) Energy Constrained Operation:
5) Limited Security:
6) Less Human Intervention:



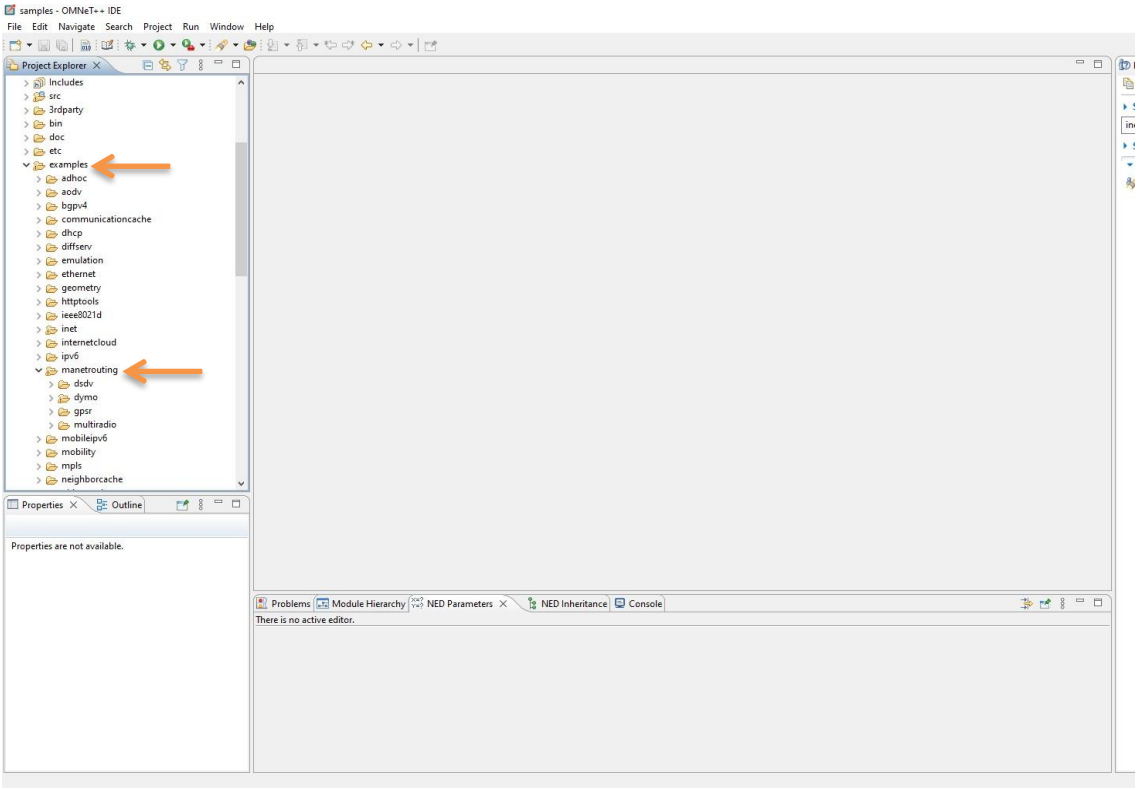
A Typical example of MANET

We create an MANET using Omnet++ and INET through the following steps

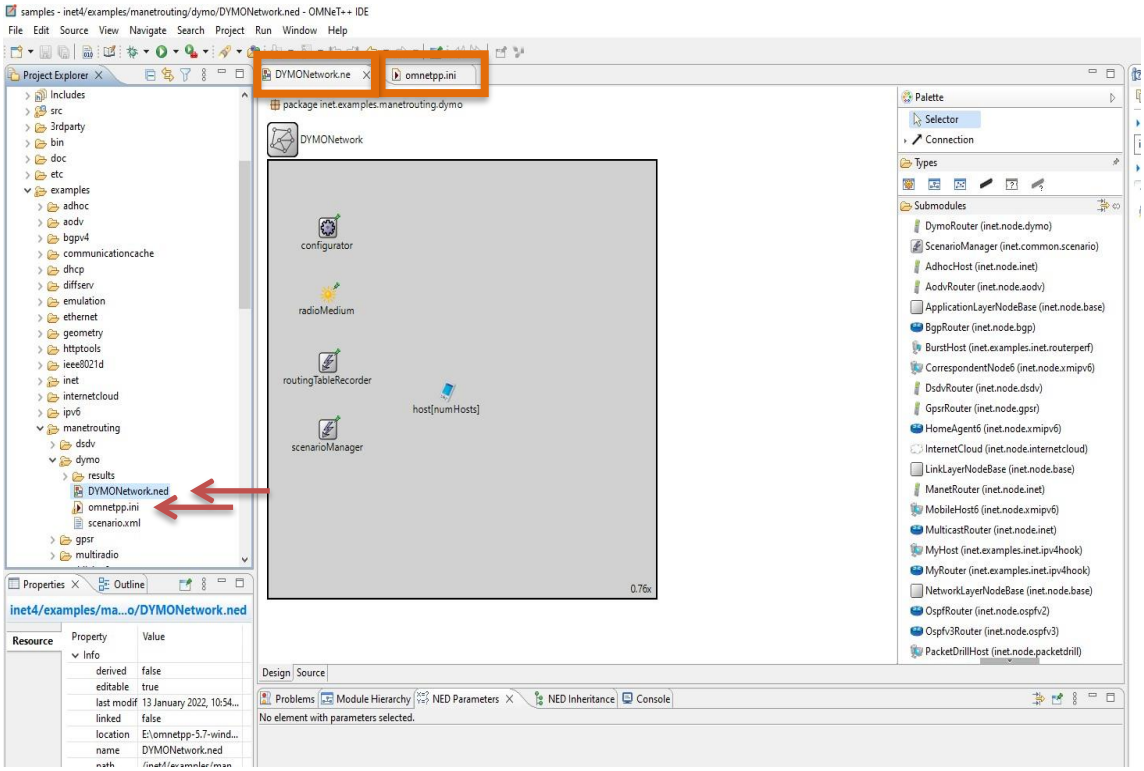
Step 1: Open the Omnet++ software and click on inet4 folder



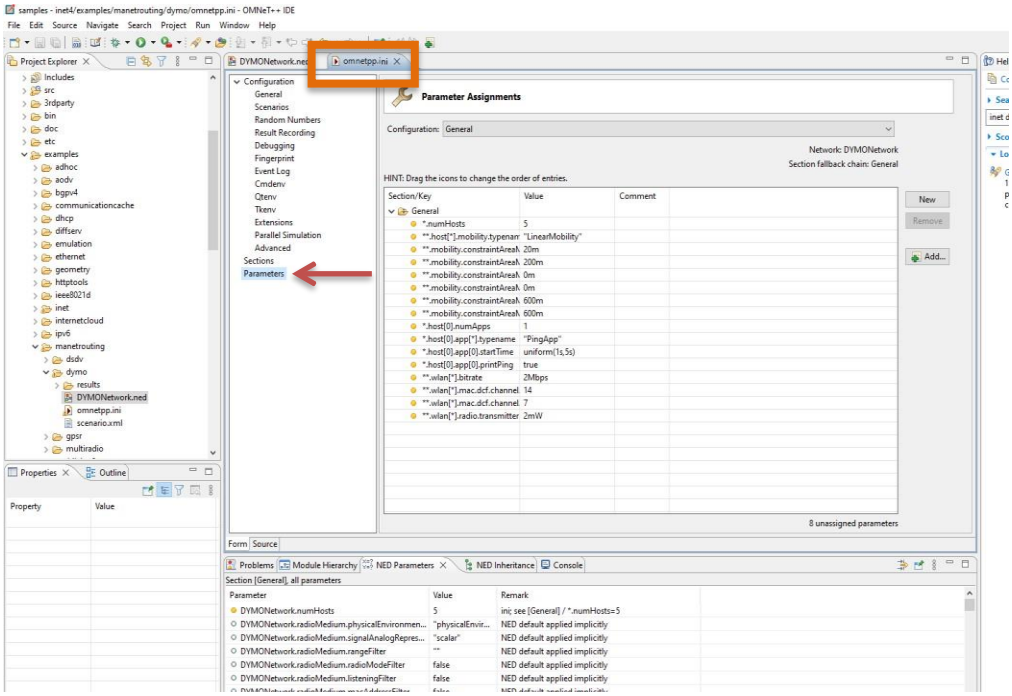
Step 2: Now select the examples folder and then in that folder select manetrouting folder



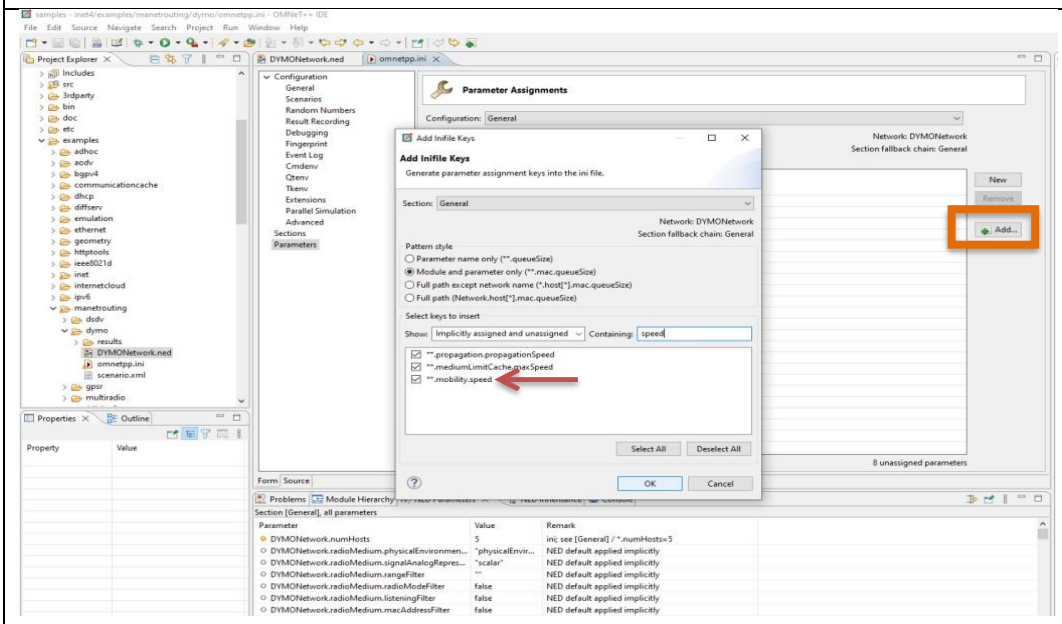
Step 3 : In manetrouting folder click dymo folder and then load the DYMONetwork.ned and omnetpp.ini files by double clicking



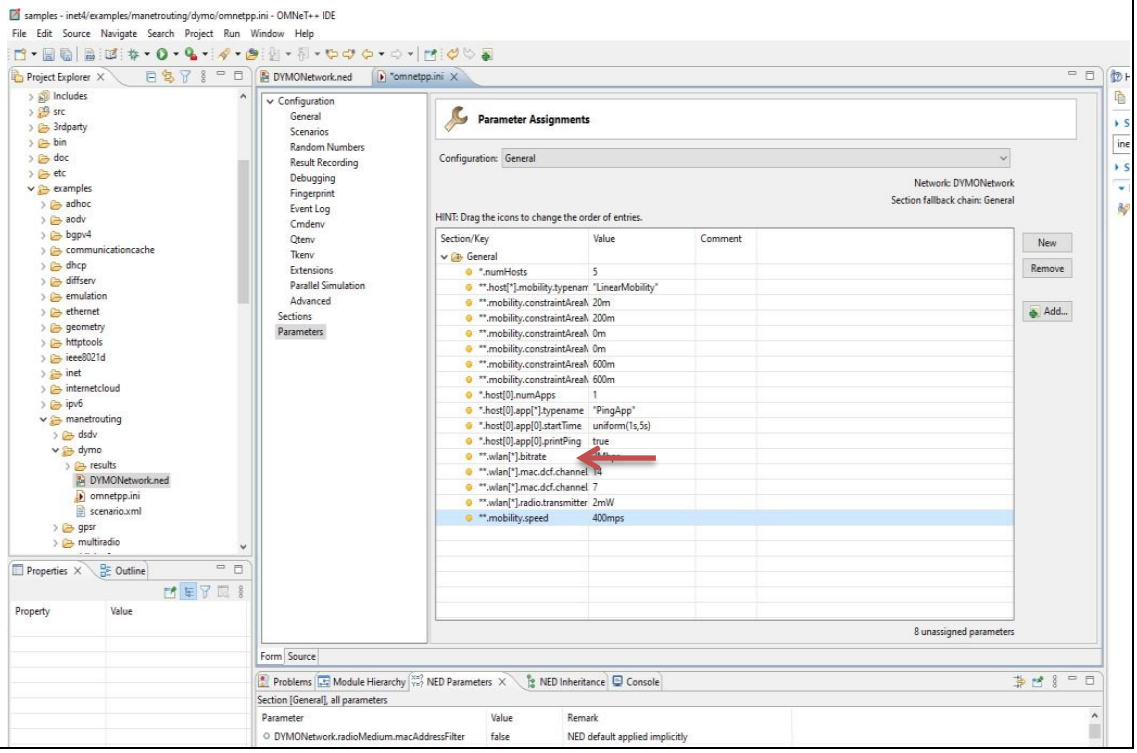
Step 4: Select omnetpp.ini file and click on parameters, we need to add mobility to the nodes



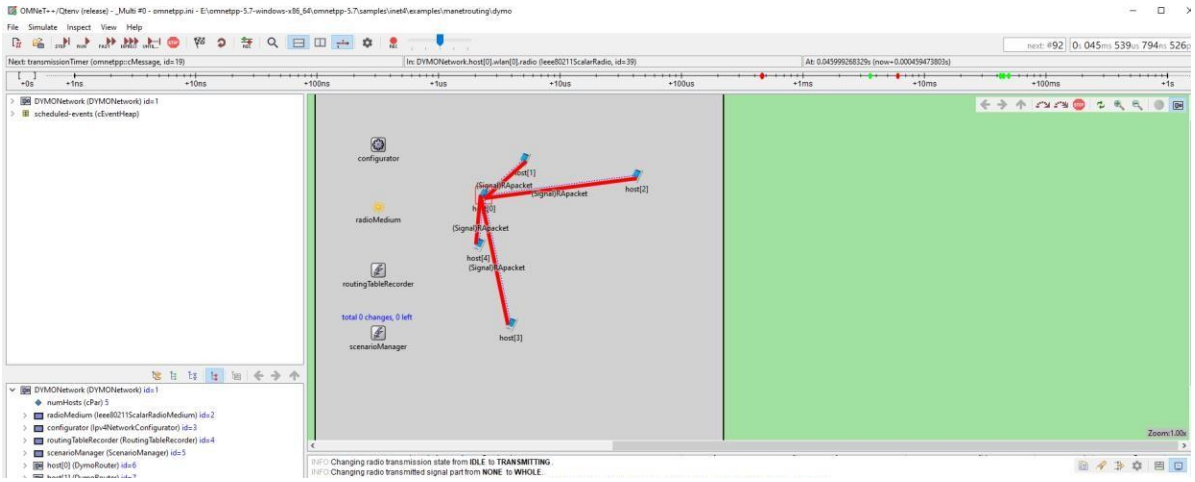
Step 5: For adding a new parameter click on add button and add the parameter **.mobility.speed



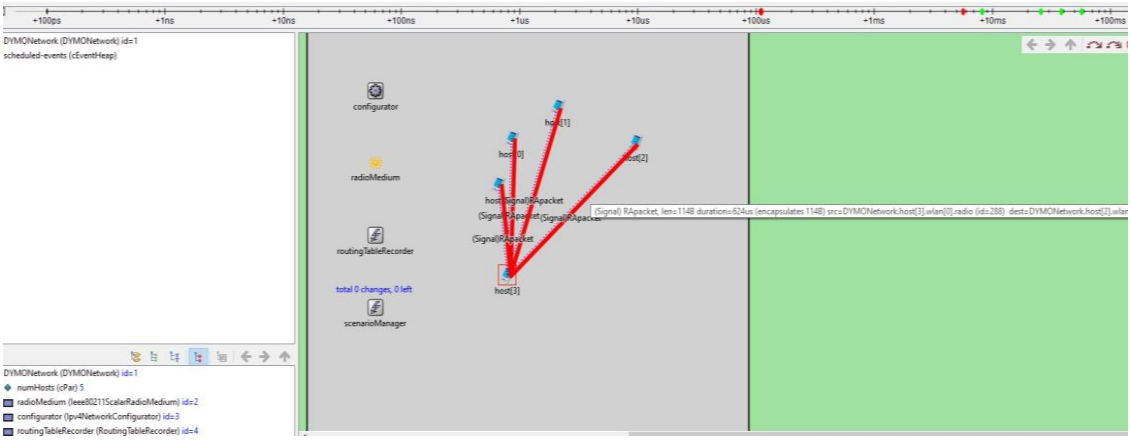
Step 6: Set the value for **.mobility.speed = 400mps



Step 7: Now we run the simulation with 5 mobile hosts forming MANET and get the following output



Since the nodes have mobility, after sometime their positions would change and we get



Hence the given MANET has been simulated with 5 hosts

For video demonstration of the given practical click on the link below or Scan the QR-code

https://youtu.be/zzN_IX1hAxs

Create MAC protocol simulation implementation for wireless sensor Network.

Aim:

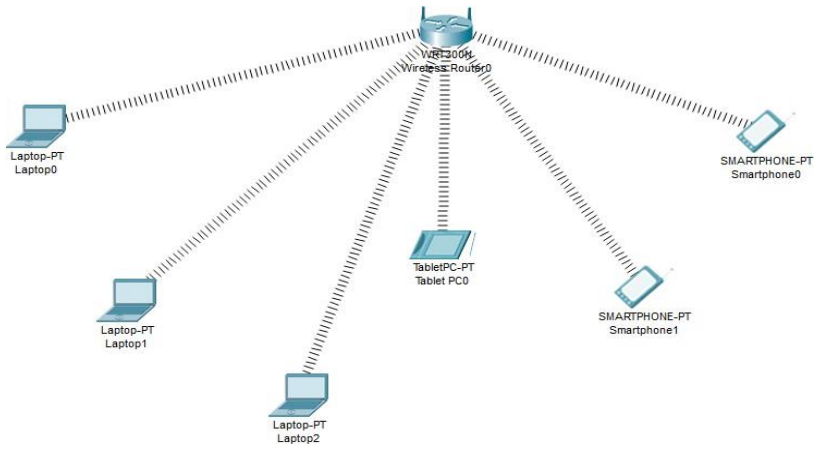
To create MAC protocol simulation for a Wireless Sensor network

Software Used:

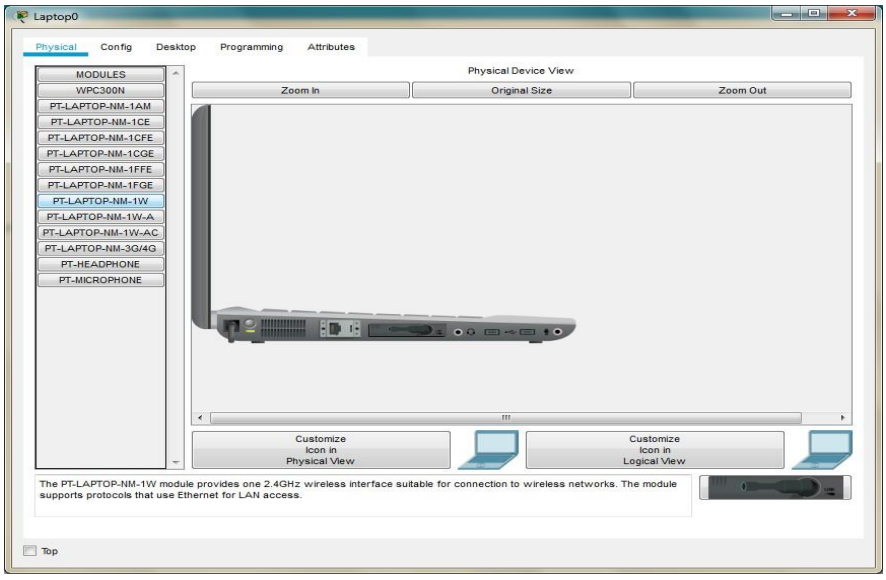
Cisco Packet Tracer 7.2.0.0226

Theory:

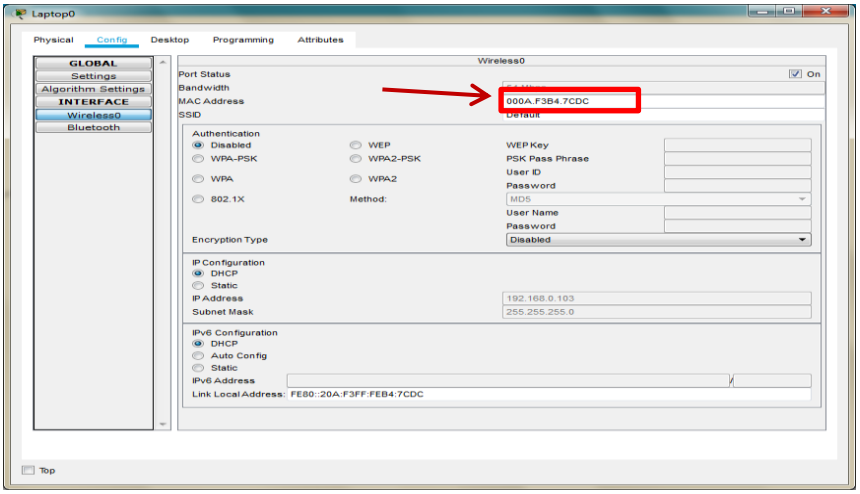
Consider the following topology, we use some wireless hosts instead of Wireless sensors. The working in both the cases is same

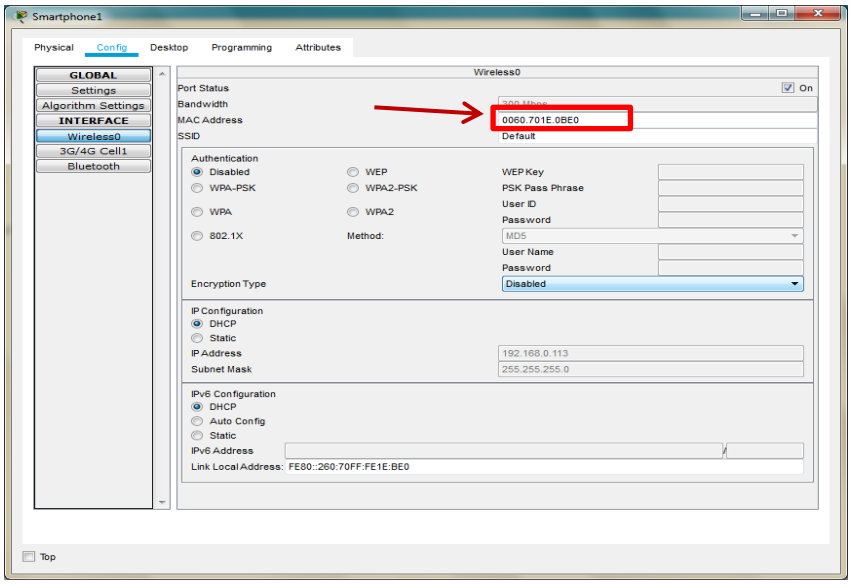


Smart phones and Tablet have a wireless interface by default, while the laptop does not has awireless interface, we need to add the interface in all the laptops, adding the wireless interface to each Laptops as follows



Copy the MAC address of each component as follows

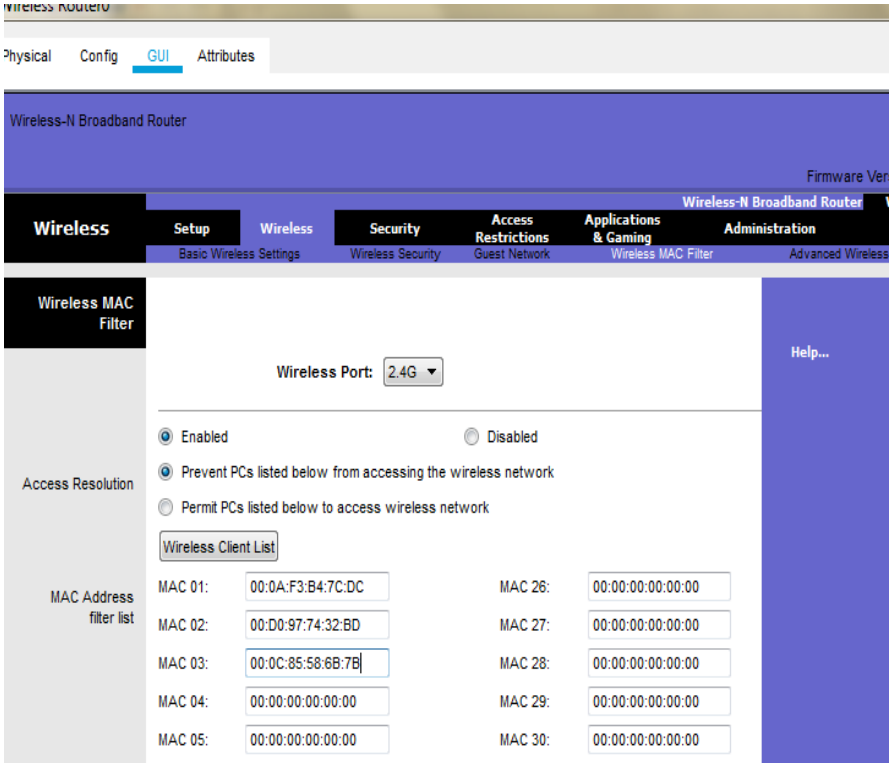




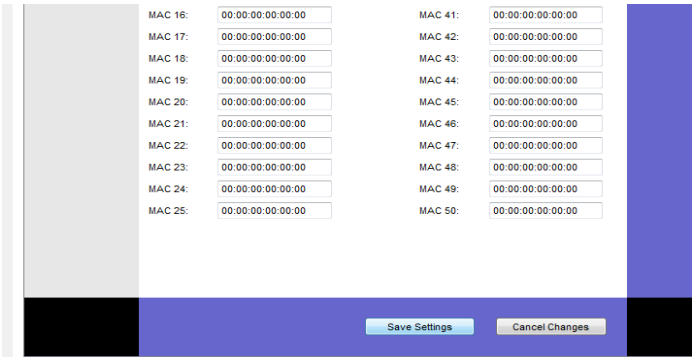
We note the following MAC addresses and convert them to the followingform

Component	MAC Address	Converted MAC address
Laptop0	000A.F3B4.7CDC	00:0A:F3:B4:7C:DC
Laptop1	0001.4269.6539	00:01:42:69:65:39
Laptop2	0060.5CB8.B919	00:60:5C:B8:B9:19
TabletPC	000C.8558.6B7B	00:0C:85:58:6B:7B
SmartPhone0	00D0.9774.32BD	00:D0:97:74:32:BD
SmartPhone1	0060.701E.0BE0	00:60:70:1E:0B:E0

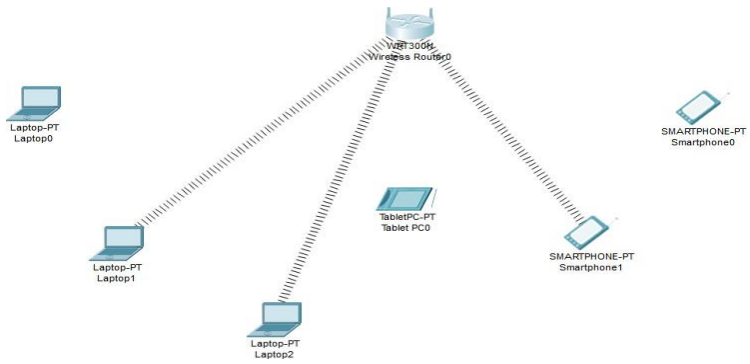
Now we add few addresses in the wireless MAC filter of the Wireless Routerand then use the given options for either allow or deny the Wireless access



As seen in above screen shot we add the MAC address of Laptop0, TabletPC, SmartPhone0 in the list so as to deny them accessing the Wireless network and then save the settings

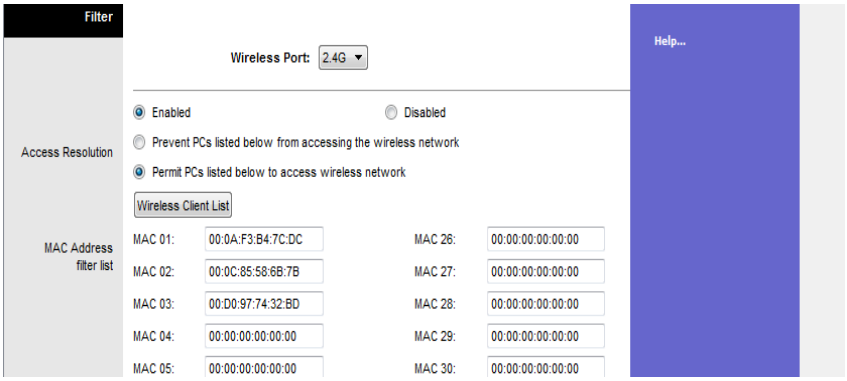


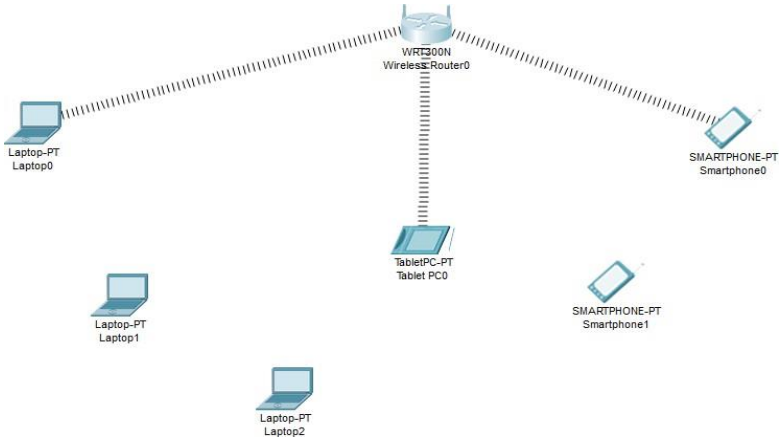
The result so obtained is as shown; the three devices denied any wireless connectivity



Similarly we can change the setting so that the above devices get wireless connectivity and the remaining devices do not get the wireless connectivity

And save the setting and get the following





For the video demonstration of the above practical click on the link below or scan the QR-code

https://youtu.be/0FEJF97K4uE

Simulating Directional Antenna

Aim: To Simulate Directional Antenna with the help of a home automation system.

Theory:

A directional antenna is a type of antenna that focuses its energy in a specific direction. Unlike omnidirectional antennas, which emit radio waves in all directions equally, directional antennas concentrate their radiated power into a narrower beam. This property makes them particularly useful for point-to-point communication over longer distances.

Directional antennas come in various designs, but they typically feature elements such as reflectors, directors, and driven elements. These elements work together to shape and direct the emitted radio waves in a specific direction, enhancing the antenna's gain and efficiency.

There are several types of directional antennas, including:

1. Yagi-Uda Antenna: This is one of the most common types of directional antennas. It consists of a driven element (a dipole), one or more directors placed in front of the driven element, and a reflector behind it. Yagi-Uda antennas are widely used in applications such as television reception and point-to-point communication.

2. Parabolic Dish Antenna: Parabolic antennas use a curved reflector dish to direct radio waves towards a focal point where the feed element (such as a dipole or a horn antenna) is located. These antennas are highly directional and are commonly used for satellite communication, microwave links, and radio astronomy.

3. Horn Antenna: Horn antennas have a flared shape that allows them to direct radio waves in a specific direction. They are often used in radar systems, microwave communication, and antenna measurement applications.

4. Patch Antenna: Patch antennas, also known as microstrip antennas, consist of a flat metallic patch suspended above a ground plane. They are commonly used in wireless communication systems, such as Wi-Fi routers and RFID devices.

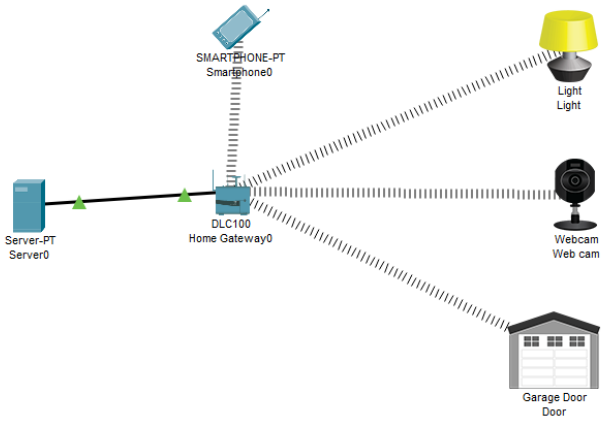
Directional antennas offer several advantages, including increased range, improved signal quality, and reduced interference from unwanted sources. However, their directional nature also means that they must be precisely aimed at the target to achieve optimal performance.

Home Automation using Directional Antenna

Simulating a directional antenna with the aid of a home automation system involves utilizing smart technology to mimic the functionality of a physical directional antenna. This can be achieved by employing smart devices equipped with wireless

communication capabilities, such as smart bulbs or plugs, as virtual signal transmitters.

Through the home automation system, users can control the activation and deactivation of these smart devices in different locations within the home. By strategically activating these devices in specific areas, users can simulate the directional transmission of signals towards desired locations.

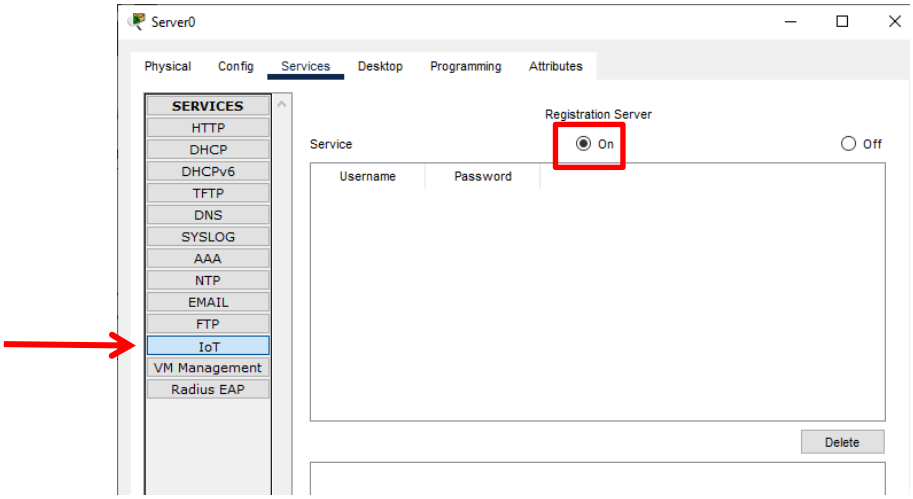


In the above example we use a mobile phone to control the a lamp, a web cam and garage door

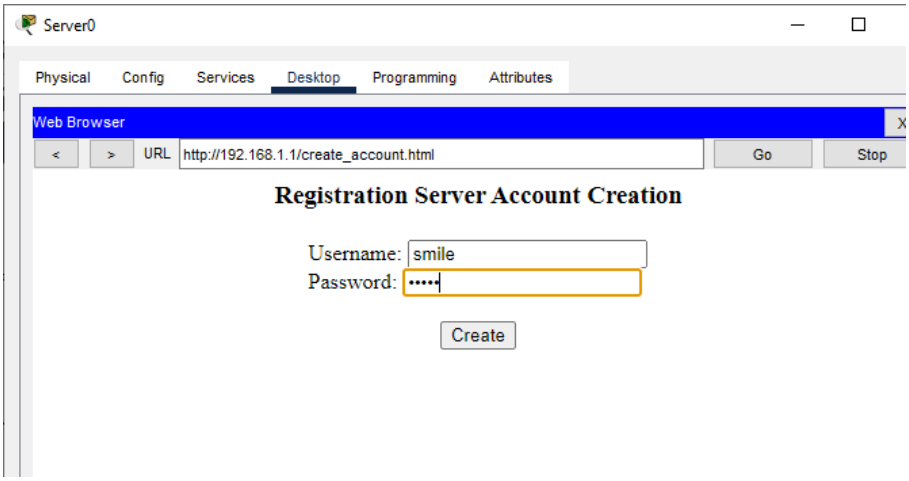
We assign the following IP addresses to the devices

Device	IP address
Server	192.168.1.1
SmartPhone	192.168.1.2
Light	192.168.1.3
Webcam	192.168.1.4
Garage Door	192.168.1.5

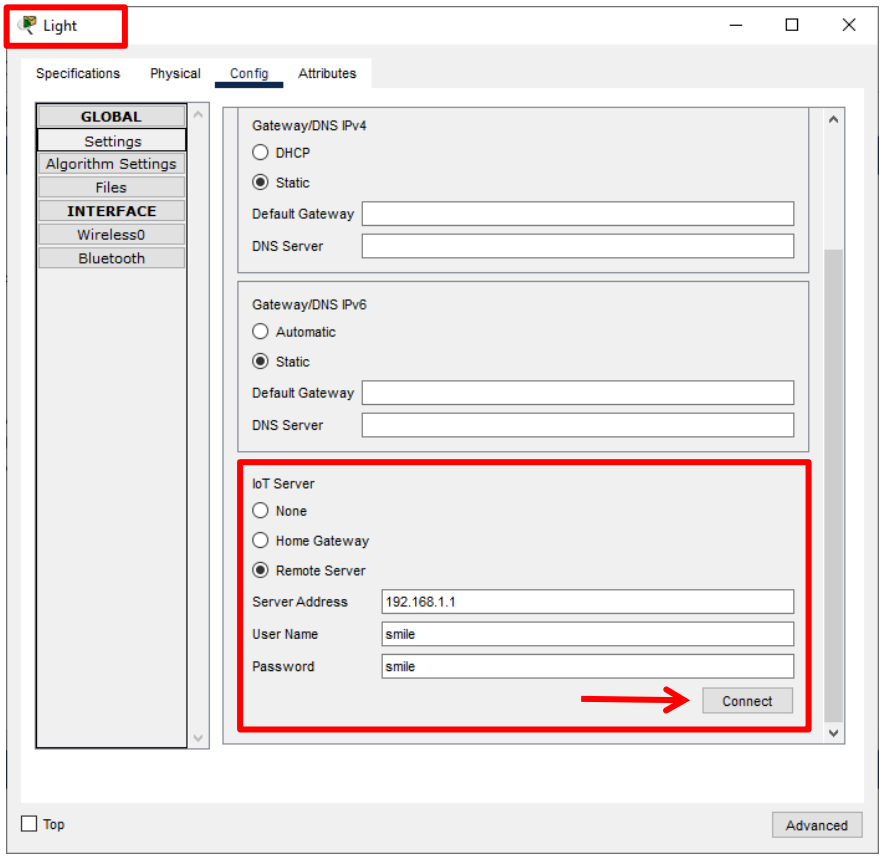
We turn ON the IoT services on the Server



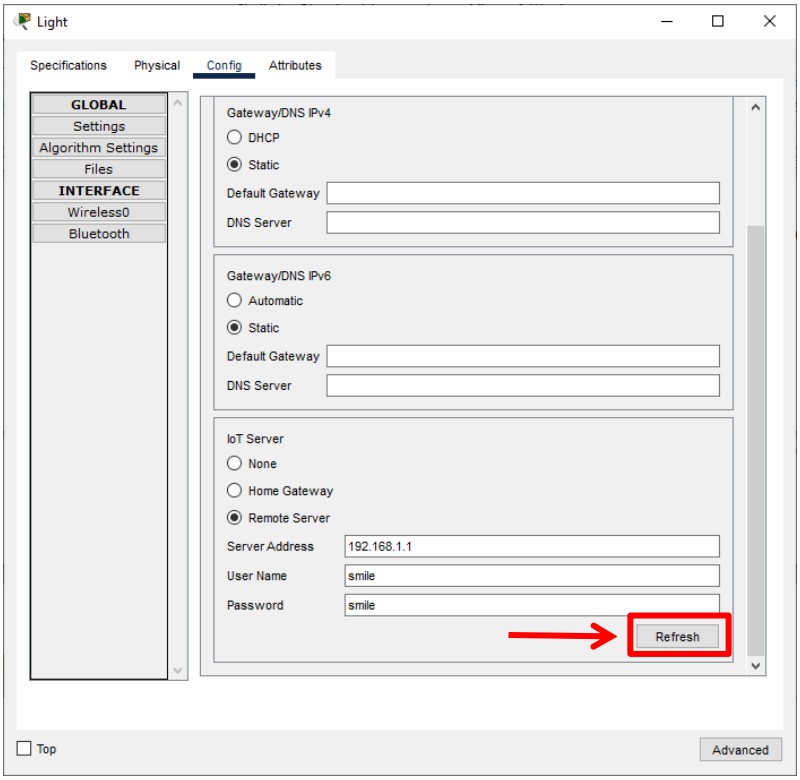
Create an account by logging through the browser from the server



Add the remote server in all the IoT devices

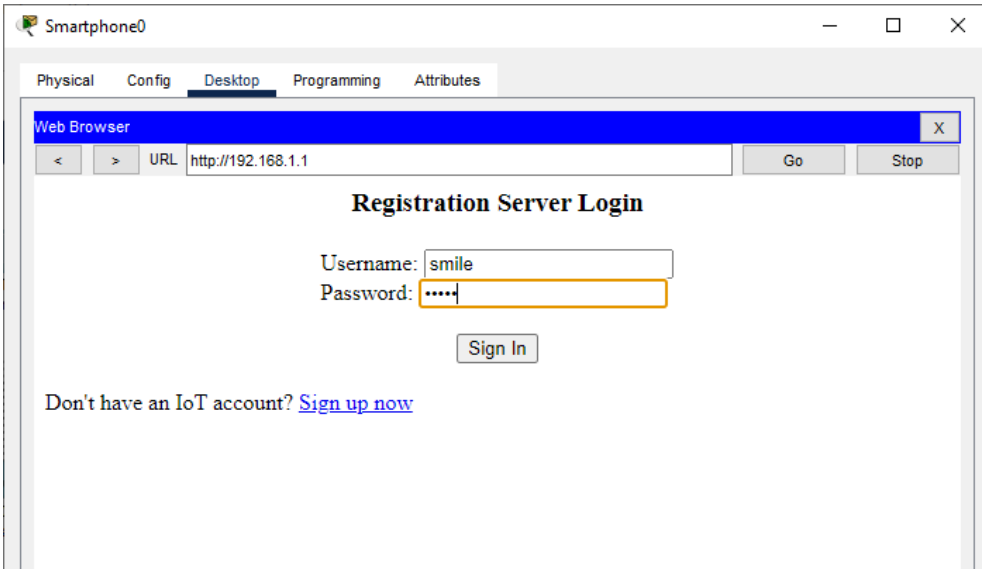


And then press the connect button, the connect button becomes the refresh button

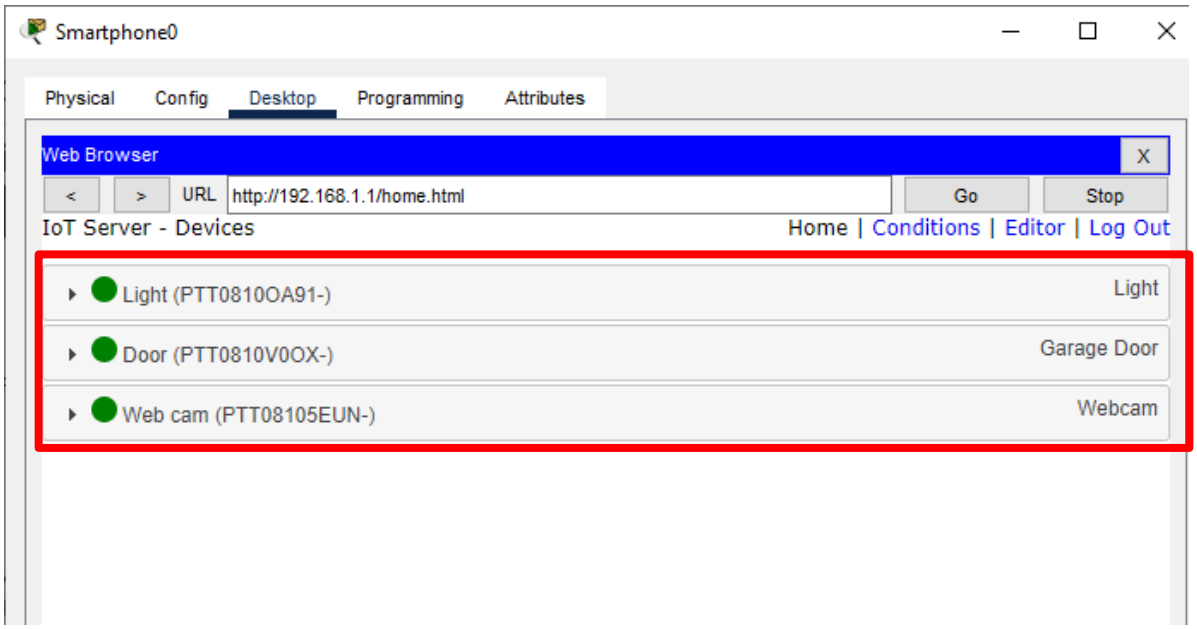


We do the above for the other two devices i.e webcam and the Garage door

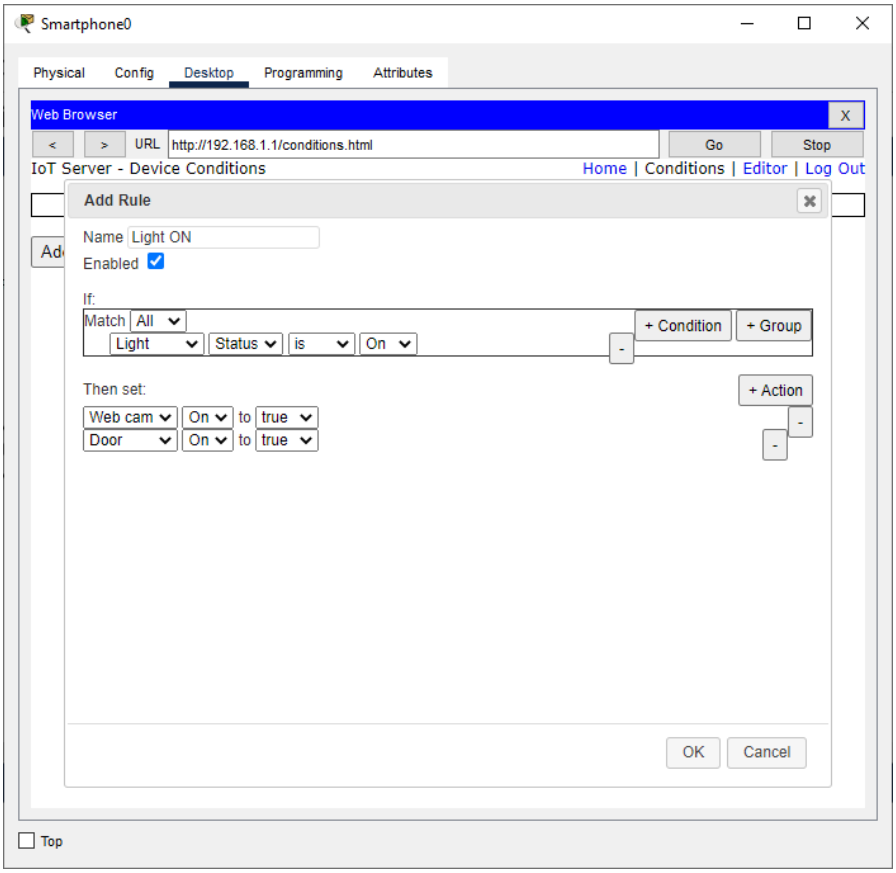
Next we login through the Smartphone browser into the server

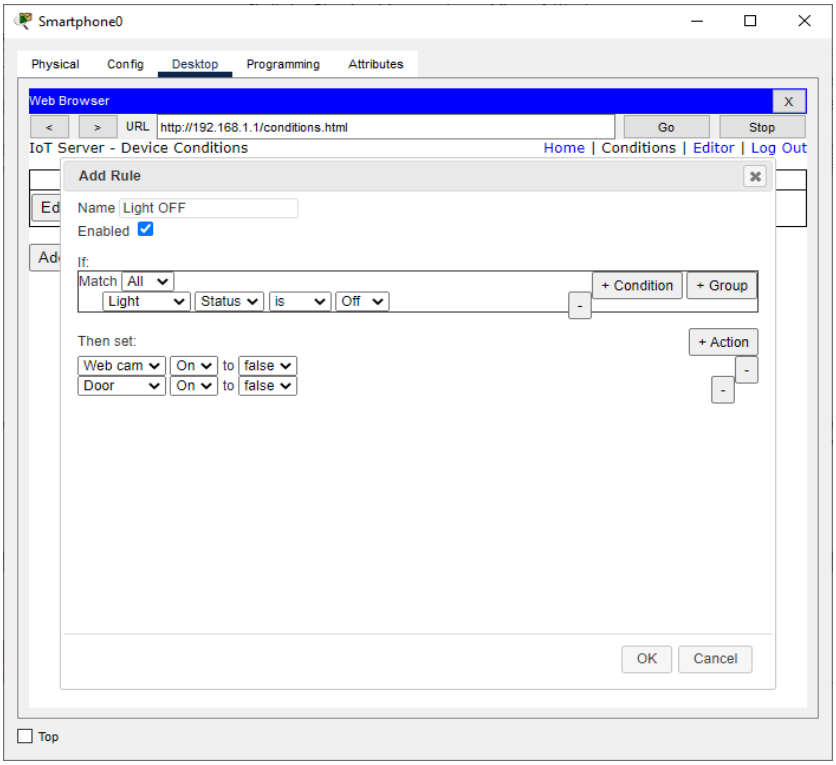


After successful login the three IoT devices are displayed and they can be controlled from the smart phone

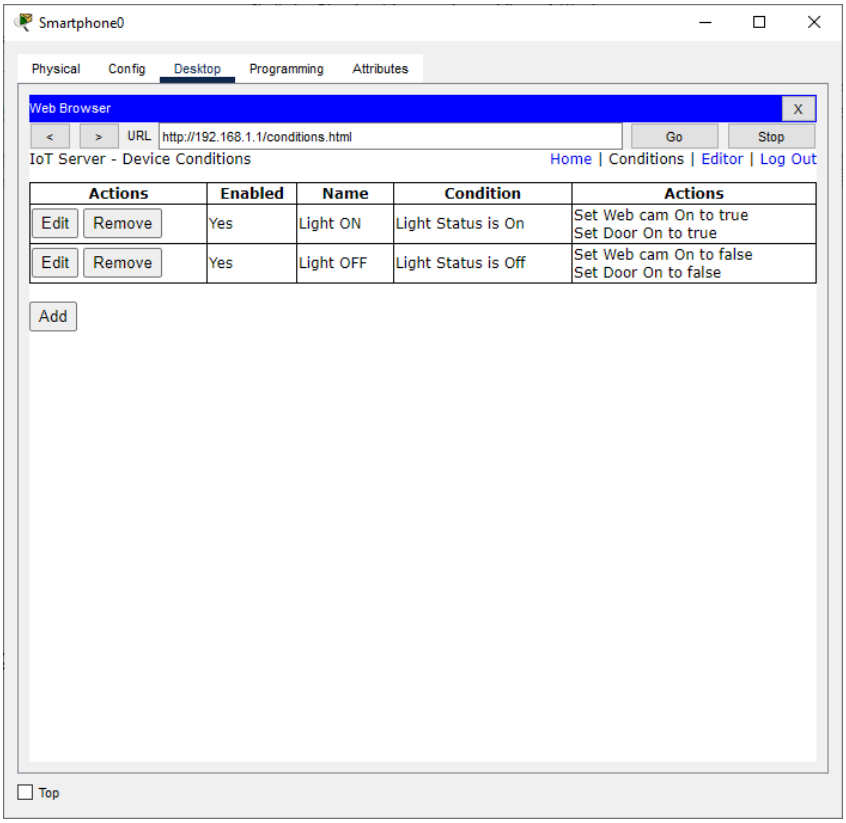


Now we apply some conditions for turning the webcam ON and OFF as well as opening and closing of Garage Door by turning the light ON and OFF

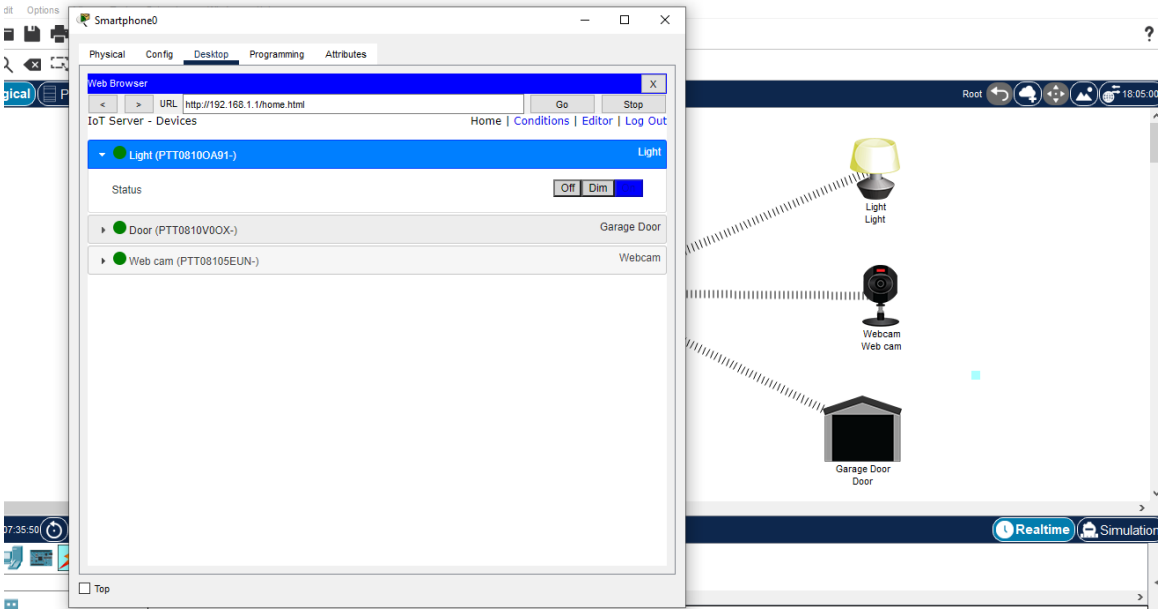




We get the following rules



We verify the rules



Hence the given simulation has been studied and verified

For the video demonstration of the practical, click on the link below or scan the QR-code

https://youtu.be/ic0H58ZJLNU